

VisMorph: A Live Programming Environment for Shape-Adaptive Computers

Yusuke Izawa*
Tokyo Metropolitan University
Tokyo, Japan
yizawa@acm.org

Junichiro Kadomoto*
The University of Tokyo
Tokyo, Japan
kadomoto@mtl.t.u-tokyo.ac.jp

Hidetsugu Irie
The University of Tokyo
Tokyo, Japan
irie@mtl.t.u-tokyo.ac.jp

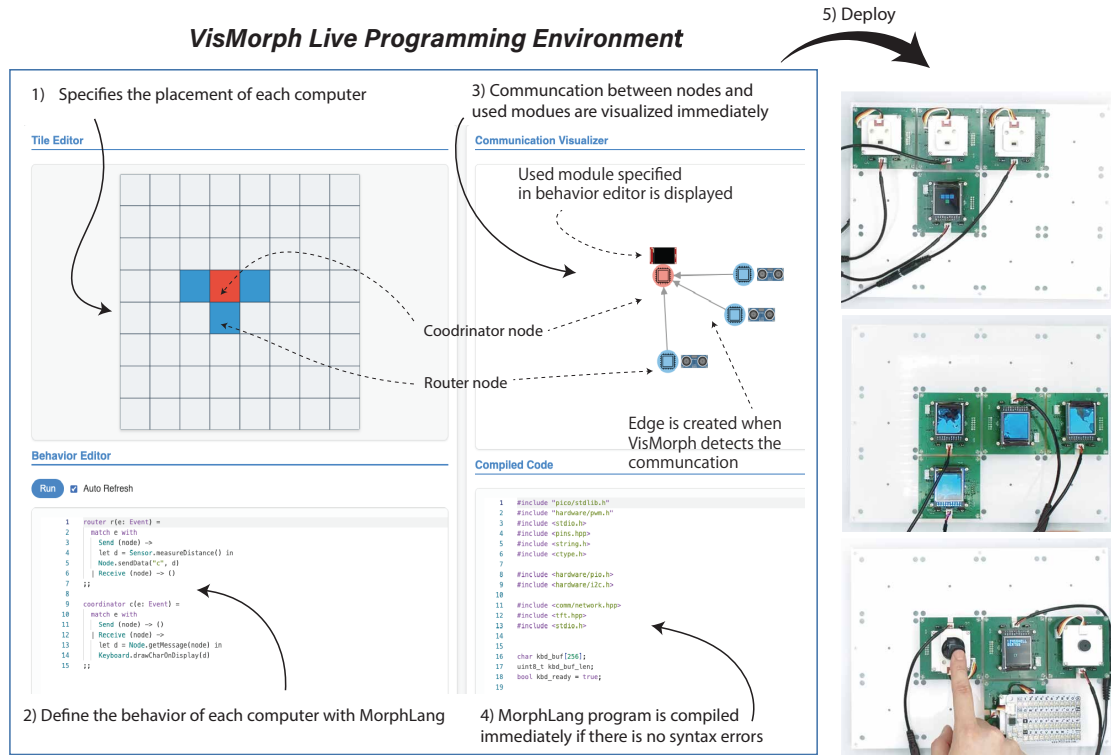


Figure 1: VisMorph is a live programming environment for a shape-changing interface that uses shape-adaptive computers.

Abstract

We present VisMorph, a live programming environment for shape-adaptive computers—systems composed of microscopic units that wirelessly coordinate to change shape and function. VisMorph supports intuitive application development for such systems through real-time visualization, interactive feedback, and a declarative programming language called MorphLang. It allows users to define physical layouts, specify behaviors, and visualize communication

on a 2D grid, making development more intuitive and efficient. VisMorph streamlines the process of programming dynamic, reconfigurable interfaces, enabling advanced workflows for shape-changing systems.

CCS Concepts

• **Software and its engineering** → **Integrated and visual development environments.**

Keywords

Live programming environment, shape-adaptive computer, shape-changing interface

ACM Reference Format:

Yusuke Izawa, Junichiro Kadomoto, and Hidetsugu Irie. 2025. VisMorph: A Live Programming Environment for Shape-Adaptive Computers. In *The 38th Annual ACM Symposium on User Interface Software and Technology (UIST Adjunct '25)*, September 28–October 01, 2025, Busan, Republic of Korea. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3746058.3758364>

*Both authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

UIST Adjunct '25, Busan, Republic of Korea

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2036-9/25/09

<https://doi.org/10.1145/3746058.3758364>

1 Introduction

Shape-adaptive computers [2, 9] are an emerging system consisting of wirelessly connected computing devices capable of dynamic shape morphing. As the complexity of these systems grows, programming an application running on them becomes a significant challenge. Since existing programming languages/environments/tools are oriented to traditional computers, they lack the dynamic feedback and morph-aware interaction features necessary for effective development on such computers.

We propose *VisMorph*, a live programming environment [4, 5, 10–12] designed to support application development for morphing interfaces. For supporting morph-awareness, we realize the combination of a traditional text-based editor and a 2D-grid layout editor designed for 2D-spatial shape-adaptive computers as a live programming environment. It adapts MorphLang [7], a functional reactive language tailored for these systems.

2 Related Work

VisMorph focuses on designing and debugging applications consisting of interacting devices each other. In this context, it is aligned with the field of live programming. *Live programming* is a paradigm where a program is immediately executed after the edit and returns the feedback to the programmer [11]. In particular, *live programming environments* are the systems that realize live programming [4, 5, 10, 12, 12].

3 VisMorph

VisMorph is a live programming environment dedicated to shape-adaptive computers. Particularly, VisMorph targets the application development of shape-adaptive computers running on a 2D-grid environment. Therefore, VisMorph needs editors to decide the layout of the computers in addition to a program editor.

VisMorph integrates a *layout editor*, *behavior editor*, and *real-time visualizer*, enabling rapid, iterative development with immediate feedback. When these inputs are successfully harmonized, *compiled code pane* shows the resulting compiled code. In particular, the layout editor lets users arrange nodes in a 2D grid and assign roles (e.g., coordinator or router). The behavior editor supports MorphLang for defining communication and behavior logic; includes syntax checking. The real-time visualizer shows real-time communication between nodes, helping users debug and understand system behavior. Finally, the compiled code pane displays code generated from MorphLang, ready for deployment.

3.1 Workflow in VisMorph

We illustrate the workflow of application software development in VisMorph with respect to the Figure 1.

1) Define the Layout. In the Tile Editor, users arrange nodes on a 2D grid, specifying *coordinator nodes* for data distribution and *router nodes* for relaying information. **2) Describe the Behavior.** In the Behavior Editor, users define node behaviors using MorphLang. Physical details are abstracted and later resolved by the compiler using layout data. **3) Visualize Interactions.** The Communication Visualizer provides real-time feedback on node communication based on current layout and behavior definitions. **1)–3) Refine Placement and Behavior.** If results are unsatisfactory, users can

revise layouts or behaviors. Changes are instantly reflected in the Communication Visualizer for immediate feedback. **4) Compile Code.** The Compiled Code pane shows generated code ready for deployment, ensuring correctness before transferring to hardware. **5) Deploy Code.** After compilation, users deploy the code to shape-adaptive computers and verify system behavior on actual devices.

4 Visualization Examples

We demonstrate VisMorph’s capabilities using CommuTiles [3], a shape-changing system built on modular tiles.

In Figure 1, the application uses distance sensors and a display to visualize spatial information. In VisMorph, users define node positions in the Tile Editor and behaviors in the Behavior Editor. Using MorphLang, router nodes collect sensor data and forward it to the coordinator, which processes and displays the data. The coordinator (red-colored tile) acts as a display, while routers (blue-colored tiles) serve as sensors. Communication is visualized as edges connecting routers to the coordinator.

Other applications, such as the world-map display and heterogeneous devices, can be developed through VizMorph, examples of which are shown on the right side of Figure 1, moved to the middle and bottom.

5 Challenges and Future Directions

While VisMorph offers editors and real-time visualization, it is not yet a fully integrated development environment like Quartus Prime [6] or Arduino [1]. It currently lacks features such as direct deployment of compiled code to devices. Enhancing VisMorph into a complete IDE would further streamline the development process and improve usability.

VisMorph should also accommodate smaller, more granular shape-adaptive computers. Prior work has shown the feasibility of integrating microcontrollers and wireless transceivers into chips as small as $5.3\text{ mm} \times 2.6\text{ mm}$ [8]. This enables applications with far more nodes than in the current CommuTiles setup. To address this, VisMorph should be extended to support large-scale configurations, 3D spatial layouts, and real-time responsiveness at scale.

6 Conclusion

We introduced VisMorph, a live programming environment for shape-adaptive computers. By combining layout editing, behavior specification in MorphLang, and real-time visualization, VisMorph simplifies the development of shape-changing applications. Although already effective for prototyping, it can be further improved with features like direct code deployment and broader hardware support. As shape-adaptive systems scale in complexity and granularity, VisMorph has the potential to evolve into a powerful, full-featured platform for adaptive computing.

Acknowledgments

This work was partially supported by JSPS KAKENHI Grant Nos. JP23H00465, JP23K16857, JP24K23866, and JP25K21180. We thank Masato Goto for the advice on the development of the visualization examples.

References

- [1] Alessandro D'Ausilio. 2012. Arduino: A low-cost multipurpose lab equipment. *Behavior research methods* 44 (2012), 305–313.
- [2] S.C. Goldstein, J.D. Campbell, and T.C. Mowry. 2005. Programmable matter. *Computer* 38, 6 (2005), 99–101. doi:10.1109/MC.2005.198
- [3] Masato Goto, Ibuki Sugiyama, Kenta Higuchi, Toshiki Goto, Junichiro Kadomoto, Hidetsugu Irie, and Shuichi Sakai. 2024. CommuTiles: Shape-Changeable Modular Computer System Using Proximity Wireless Communication. In *Extended Abstracts of the 2024 CHI Conference on Human Factors in Computing Systems (CHI EA '24)*. Association for Computing Machinery, New York, NY, USA, Article 395, 5 pages. doi:10.1145/3613905.3648669
- [4] Philip J. Guo. 2013. Online Python Tutor: Embeddable Web-Based Program Visualization for Cs Education. In *Proceeding of the 44th ACM Technical Symposium on Computer Science Education (Denver, Colorado, USA) (SIGCSE '13)*. Association for Computing Machinery, New York, NY, USA, 579–584. doi:10.1145/2445196.2445368
- [5] Brian Hempel, Justin Lubin, and Ravi Chugh. 2019. Sketch-n-Sketch: Output-Directed Programming for SVG. In *Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology (New Orleans, LA, USA) (UIST '19)*. Association for Computing Machinery, New York, NY, USA, 281–292. doi:10.1145/3332165.3347925
- [6] Intel. 2025. Quartus® Prime Design Software. <https://www.intel.com/content/www/us/en/products/details/fpga/development-tools/quartus-prime/resource.html> accessed Jan. 23 2025.
- [7] Yusuke Izawa, Junichiro Kadomoto, Hidetsugu Irie, and Shuichi Sakai. 2024. Designing a Reactive Programming Language for Shape-Adaptive Computers. In *2024 31st Asia-Pacific Software Engineering Conference (APSEC)*. 452–456. doi:10.1109/APSEC65559.2024.00058
- [8] Junichiro Kadomoto, Hidetsugu Irie, and Shuichi Sakai. 2020. Design of Shape-Changeable Chiplet-Based Computers Using an Inductively Coupled Wireless Bus Interface. In *2020 IEEE 38th International Conference on Computer Design (ICCD)*. 589–596. doi:10.1109/ICCD50377.2020.00103
- [9] Junichiro Kadomoto, Takuya Sasatani, Koya Narumi, Naoto Usami, Hidetsugu Irie, Shuichi Sakai, and Yoshihiro Kawahara. 2021. Toward Wirelessly Cooperated Shape-Changing Computing Particles. *IEEE Pervasive Computing* 20, 3 (2021), 9–17. doi:10.1109/MPRV.2021.3086035
- [10] Khan Academy. 2025. Intro to JS: Drawing & Animation. <https://www.khanacademy.org/computing/computer-programming/programming>. Accessed Jan. 2025.
- [11] Sean McDirmid. 2007. Living it up with a live programming language. *SIGPLAN Not.* 42, 10 (Oct. 2007), 623–638. doi:10.1145/1297105.1297073
- [12] Sean McDirmid. 2013. Usable live programming. In *Proceedings of the 2013 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software (Indianapolis, Indiana, USA) (Onward! 2013)*. Association for Computing Machinery, New York, NY, USA, 53–62. doi:10.1145/2509578.2509585